Theory of Computation

GATE CSE 2027 · complete formula, rule, method & trap sheet

78 reference boxes · **8** chapters ·

Finite Automata (DFA & NFA) | Regular Expressions & Closure Properties | Context-Free Grammars | Pushdown Automata | Pumping Lemma & Language Properties | Turing Machines & Undecidability | Problem → Standard Method Playbook | Last-minute one-liners

1 Finite Automata (DFA & NFA)

What is a DFA?

NOTES

A **Deterministic Finite Automaton** is a 5-tuple `(Q, Σ, δ, q0, F)` : a finite set of states Q, an alphabet Σ, a transition function δ (exactly one arrow per symbol from every state), a start state q0, and accepting states F. It reads the input left to right, one symbol at a time, and accepts if it ends in an accepting state.

Example: strings over {0,1} ending in "01"

DIAGRAM



NOTES

q0 = "haven't seen anything useful yet", q1 = "just saw a 0", q2 (double circle = accepting) = "just completed ...01". Any further symbol from q2 loops back on itself only if it re-forms "01" — here simplified as a self-loop for both symbols for illustration; a fully correct machine would route q2 on '0' back to q1 and on '1' back to q0. **GATE tip:** always trace 2-3 test strings by hand ("101" → q0-1→q0-0→q1-1→q2 ✓ accepted) rather than trusting a diagram at a glance.

NFA — nondeterminism

NOTES

An **NFA** allows zero, one, or many outgoing arrows per symbol from a state (and optionally ε-moves that consume no input). It accepts a string if **at least one** path through the machine ends in an accepting state.

COMMON TRAP

Key theorem

Every NFA has an **equivalent DFA** (they accept exactly the same class of languages — regular languages). NFAs are just a convenient shorthand; DFAs are what actually run in constant memory per step.

NFA → DFA: the subset construction

NOTES

Each DFA state = a **set** of NFA states reachable so far. Start state = ε-closure of the NFA's start state. On each symbol, take the union of where every state in the current set can go.

WORKED EXAMPLE

Worked numerical

Q: NFA: states {A,B}, A is start, B is accepting. δ(A,0)={A,B}, δ(A,1)={A}, δ(B,0)=∅, δ(B,1)={B}. Convert to a DFA (list only reachable states).
A:

```
Start = {A}
{A} --0--> {A,B}          {A} --1--> {A}
{A,B} --0--> {A,B}∪∅ = {A,B}      {A,B} --1--> {A}∪{B} =
{A,B}
```

Reachable DFA states: `{A}` (non-accepting, start), `{A,B}` (accepting, since it contains B). Only **2 DFA states** needed — even though the "obvious" powerset has $2^2=4$ subsets, only the reachable ones matter.

DFA Minimization

NOTES

Different DFAs can accept the same language; the **minimal DFA** has the fewest states. Method: start by splitting states into {accepting} and {non-accepting}, then repeatedly split a group if two states in it go to *different* groups on some symbol. Stop when no group splits further — the final groups are the minimized states.

WORKED EXAMPLE

Worked numerical

Q: A DFA has states A(start),B,C,D,E(accepting) with: δ(A,0)=B, δ(A,1)=C, δ(B,0)=A, δ(B,1)=D, δ(C,0)=E, δ(C,1)=A, δ(D,0)=E, δ(D,1)=C, δ(E,0)=D, δ(E,1)=B. Minimize it.
A:

```
Initial partition: P0 = {A,B,C,D} (non-accept), {E} (accept)

Round 1 — check each state's behavior against groups
{A,B,C,D} / {E}:
  A: 0→B(grp1) 1→C(grp1)  → pattern (1,1)
  B: 0→A(grp1) 1→D(grp1)  → pattern (1,1)
  C: 0→E(grp2) 1→A(grp1)  → pattern (2,1)   ← different!
  D: 0→E(grp2) 1→C(grp1)  → pattern (2,1)   ← different!
So split: {A,B} stay together, {C,D} split off, {E} alone.
New partition: {A,B}, {C,D}, {E}

Round 2 — recheck with 3 groups:
  A: 0→B(same grp as A) 1→C(grp2)
  B: 0→A(same grp) 1→D(grp2)      → A,B still match, no
further split
  C: 0→E(grp3) 1→A(grp1)
  D: 0→E(grp3) 1→C(grp2)          ← C goes to grp1 on '1', D
goes to grp2 on '1' → SPLIT {C,D}!

Final partition: {A,B}, {C}, {D}, {E} → 4 states after
minimization
```

The 5-state DFA minimizes to **4 states** (only A and B are truly equivalent).

GATE FAVOURITE

GATE angle: minimum states for a language

Classic question type: "minimum number of states in a DFA accepting binary strings where the third-last symbol is 1" needs **2^k** states for "k-th last symbol" style languages (the DFA must remember the last k symbols seen, and there are 2^k possible windows). For k=3, that's **8 states**. Always ask: "how much history must this DFA remember?" — that count becomes your state count.

More DFA construction patterns — worked examples

NOTES

GATE tests a handful of recurring "language types". Here are the main families, each with the trick that builds the DFA fast.

NOTE

Type 1 — Counting mod k

Q: Construct a DFA over {0,1} accepting strings where the number of 1's is ≡ **2 (mod 3)**. How many states are needed?
A: Use one state per remainder: `S0, S1, S2` (count mod 3 so far), start = S0. On '0' stay in the same state (doesn't change the count); on '1' move to the next remainder, wrapping around: S0→S1→S2→S0. Accepting state = **S2** (remainder 2). **3 states** total — one per residue class. This "counting mod k" pattern always needs exactly **k states**.

NOTE

Type 2 — Forbidden substring

Q: Construct a DFA over {0,1} accepting strings that do **not** contain "101" as a substring. How many states?

A: States track "how much of the forbidden pattern have I just seen": `q0` =nothing useful, `q1` =just saw "1", `q2` =just saw "10", `q3` =dead (saw "101", trap forever). Transitions: q0 -1→ q1, q0 -0→ q0; q1 -0→ q2, q1 -1→ q1; q2 -1→ q3(dead), q2 -0→ q0; q3 loops on both symbols. Accepting = {q0,q1,q2} (all except the dead trap). **4 states** (3 useful + 1 dead/trap state) — this "prefix-tracking" technique works for any forbidden substring, needing (length of pattern + 1) states in the simplest case.

NOTE

Type 3 — Product construction (AND of two conditions)

Q: Construct a DFA for strings with an **even** number of 0's **and** an **odd** number of 1's.

A: Build two small DFAs separately (2 states each: even/odd count of 0's; even/odd count of 1's), then take their **Cartesian product**: each combined state = (0-parity, 1-parity), giving 2×2 = **4 states**: (even,even), (even,odd), (odd,even), (odd,odd). Accepting state = (even,odd). This "product construction" is the standard way to combine two separate regular conditions with AND — and it directly proves regular languages are closed under intersection.

ε-NFA and ε-closure

NOTES

An **ε-NFA** allows transitions that consume no input symbol at all (labeled ε). The **ε-closure** of a state is every state reachable using only ε-moves (including itself).

WORKED EXAMPLE

Worked numerical

Q: ε-NFA: states A,B,C. ε-moves: A→B, B→C. δ(C,0)={A}. Find ε-closure(A) and the DFA-equivalent start state.

A: From A you can reach B (ε), then C (ε from B) → **ε-closure(A) = {A,B,C}**. This whole set becomes the single start state of the equivalent DFA (same idea as subset construction, but every step first takes the ε-closure of wherever the input symbols land you).

Finite automata with output — Moore & Mealy machines

NOTES

Unlike a DFA (which only accepts/rejects), these machines **produce an output string**. They have no final states.

REFERENCE TABLE

	Moore machine	Mealy machine
Output depends on	the **current state** only	the **current state AND input** (output sits on the transition)
Output length for an input of length n	**Depends on the question's convention — not on Moore vs Mealy.** If the start state's output is printed before any input, a Moore machine gives n + 1; if the question says it is not printed, Moore also gives n. A Mealy machine prints one output per transition, so n. Always read how the question defines the output sequence.	
Reacts to input	one clock later	immediately

KEY POINT

Conversions. Moore → Mealy keeps the **same number of states** (move each state's output onto its incoming transitions). Mealy → Moore may need **more states** — a state must be split by the output it receives, so up to |Q| × |outputs| states. When the Moore machine's start-state output is printed, the two outputs match after dropping that first symbol.

WORKED EXAMPLE

Worked numerical — "11" detector

Q: Mealy machine, states q0 (last bit not 1) and q1 (last bit was 1): q0 −0/0→ q0, q0 −1/0→ q1, q1 −0/0→ q0, q1 −1/1→ q1. Output for input 0110111? Equivalent Moore machine size?

A:

```
Input : 0 1 1 0 1 1 1
Output: 0 0 1 0 0 1 1     (n = 7 input symbols → 7 output symbols)
Moore version needs 3 states: A (no trailing 1, out 0), B (one trailing 1, out 0), C (≥2 trailing 1s, out 1)
Under the convention that the start state's output is printed, its output on the same input has 8 symbols: 0 0010011 — the extra leading 0 is the start state's output.
```

2 Regular Expressions & Closure Properties

Regex operators

REFERENCE TABLE

Operator	Meaning	Example
Concatenation `ab`	a followed by b	`01` matches only "01"
Union `a+b` (or a|b)	a or b	`0+1` matches "0" or "1"
Kleene star `a*`	zero or more repetitions of a	`0*` matches "", "0", "00", ...
Plus `a+`	one or more repetitions (a·a*)	`1+` matches "1","11",...

MUST REMEMBER

Kleene's theorem — three ways to describe exactly the same class of languages (regular languages): **regular expressions** ⇔ **NFAs** ⇔ **DFAs**. GATE often asks you to convert between them or count states/symbols in one representation given another.

Reading tricky regexes

WORKED EXAMPLE

Worked numerical

Q: What language does `(0+1)*011(0+1)*` describe, and does it accept "10110"?

A: All binary strings that **contain "011" as a substring** somewhere. Check "10110": does it contain "011"? Positions: 1-0-1-1-0 → substring at index 1-3 is "011" → **yes, accepted**.

WORKED EXAMPLE

Worked numerical

Q: Simplify `(0*1*)*` to the smallest equivalent regex.

A: `0*1*` already generates every possible mix once you allow repeating the whole group any number of times — repeating (0*1*) just re-generates strings already reachable. So `(0*1*)* = (0+1)*` — **every string over {0,1}**. (This is a favorite GATE trick: a star-of-a-star-like pattern often collapses to "everything".)

Closure properties of regular languages

NOTES

Regular languages are closed (the result is still regular) under:

REFERENCE TABLE

Operation	Closed?
Union	✔ Yes
Intersection	✔ Yes
Complement	✔ Yes
Concatenation	✔ Yes
Kleene star	✔ Yes
Reversal	✔ Yes

GATE FAVOURITE

GATE trap

Regular languages are closed under **every** common operation — this is what makes them so "well-behaved". Context-free languages (Part 2) are **not** closed under intersection or complement — that's where GATE likes to test whether you mixed up the two classes.

More closure operations — homomorphism, quotient & traps

REFERENCE TABLE

Operation	Regular languages	How / example
Difference, symmetric difference	✔ Closed	$L_1 - L_2 = L_1 \cap L_2^c$
Prefix (Init), suffix	✔ Closed	make every state on a path to an accepting state accepting
Homomorphism h(L)	✔ Closed	h replaces each symbol by a string: h(a)=01 turns a regex a* into (01)*
Inverse homomorphism $h^{-1}(L)$	✔ Closed	{w : h(w) ∈ L}; the DFA reads h(a) in one step
Substitution by regular sets	✔ Closed	replace each symbol by a regular language
Quotient L_1 / L_2	✔ Closed for **ANY** L_2	{x : xy ∈ L_1 for some y ∈ L_2}

GATE FAVOURITE

GATE trap

Closure is only for **finitely many** operations. An **infinite union** of regular languages need not be regular (every language is the infinite union of its single-string sets). Also: a **subset** of a regular language need not be regular ($\{a^n b^n\} \subset a^*b^*$), while any **finite** language is regular, and adding/removing finitely many strings never changes regularity.

WORKED EXAMPLE

Worked numerical

Q: L = $\{a^n b^n : n \geq 0\}$. With h(a)=0 and h(b)=ε, what is h(L)? With h(a)=0 and h(b)=1?

A: h(a)=0, h(b)=ε gives $\{0^n : n \geq 0\}$ = **0*** (regular, even though L is not). h(a)=0, h(b)=1 gives $\{0^n 1^n\}$ (not regular). So the image of a *non-regular* language may or may not be regular.

Counting accepted strings — transfer matrix trick

NOTES

To count how many length-n strings a DFA accepts, build the adjacency matrix M where M[i][j] = number of symbols taking state i to state j, then **M^n**, and sum the (start, accepting) entries.

WORKED EXAMPLE

Worked numerical

Q: A 2-state DFA over {0,1}: state A (start, accepting) with δ(A,0)=B, δ(A,1)=A; state B with δ(B,0)=A, δ(B,1)=B. How many length-2 strings does it accept?

A: Adjacency matrix (rows/cols = A,B): `M = [[1,1],[1,1]]` (A→A on '1' =1 way, A→B on '0'=1 way; B→A on '0'=1, B→B on '1'=1).

```
M^2 = [[1,1],[1,1]] × [[1,1],[1,1]] = [[2,2],[2,2]]
```

Start=A, accepting={A} → answer = M^2[A][A] = **2**. (Direct check: strings "10" and "01" both end back at A — matches.)

Arden's Theorem — converting a DFA directly into a regex

NOTES

Write one equation per state: `state = Σ (incoming symbol · source state) [+ ε if it's the start state]`. Then solve using **Arden's rule**: if `R = Q + RP` (and ε ∉ P), then `R = QP*`.

WORKED EXAMPLE

Worked numerical

Q: DFA: states A (start, accepting), B. δ(A,0)=B, δ(A,1)=A, δ(B,0)=A, δ(B,1)=B. Find the regex for the accepted language using Arden's theorem.

A: Write the state equations (a state equals ε for the start state, plus one term per incoming edge):

```
A = ε + A·1 + B·0        (A is start: add ε; incoming edges: A--1-->A, B--0-->A)
B = A·0 + B·1             (incoming edges: A--0-->B, B--1-->B)

Solve B first using Arden's rule (B = A·0 + B·1  ⇒  R=B, Q=A·0, P=1):
  B = A·0·1*

Substitute into the A equation:
  A = ε + A·1 + (A·0·1*)·0
  A = ε + A·(1 + 0·1*·0)

Apply Arden's rule again (R=A, Q=ε, P = 1+0·1*·0):
  A = ε·(1 + 0·1*·0)* = (1 + 0·1*·0)*
```

Since A is the only accepting state, the final regex is `(1 + 0·1*·0)*` — this describes exactly the strings where every "0" is eventually followed (after any number of 1's) by another "0", i.e. an even number of 0's overall.

COMMON TRAP

GATE technique tip

Always solve the equation with the **most self-loops referring only to itself** first (apply Arden's rule to isolate it), then substitute backward into the remaining equations. Non-accepting states with no useful path can often be dropped early to simplify the algebra.

More regex ⇄ language identification — varied examples

NOTE

Type A — "exactly one" pattern

Q: What does `0*10*10*` describe?

A: Any string over {0,1} containing **exactly two 1's** (with any number of 0's before, between, and after). Extending the pattern (adding more "0*1" blocks) generalizes to "exactly n ones".

NOTE

Type B — union of prefixes

Q: What does `(0+1)*0(0+1)*1(0+1)* + (0+1)*1(0+1)*0(0+1)*` describe, more simply?

A: The first part = "contains a 0 somewhere before a 1 somewhere"; the second = "contains a 1 somewhere before a 0 somewhere". Together they cover every string containing **both a 0 and a 1** — simplified to just `(0+1)*0(0+1)*1(0+1)* + (0+1)*1(0+1)*0(0+1)*`, but really it's just "not all-0's and not all-1's", i.e. the complement of `0* + 1*`.

NOTE

Type C — length restriction

Q: Give a regex for binary strings of **length exactly divisible by 3** (length ≡ 0 mod 3, length can be 0).

A: Group the alphabet into blocks of 3 symbols: `((0+1)(0+1)(0+1))*`. Each repetition of the parenthesized group consumes exactly 3 symbols, so the star produces every string whose length is a multiple of 3 (including the empty string).

3 Context-Free Grammars

What is a CFG?

NOTES

A **Context-Free Grammar** is a 4-tuple `(V, Σ, R, S)` : variables V, terminals Σ, production rules R of the form `A → α` (one variable to a string of variables/terminals), and a start symbol S. "Context-free" means a variable can be replaced the same way no matter what surrounds it.

KEY POINT

Example — balanced parentheses: `S → (S)S | ε`
Leftmost derivation of "()()" :

```
S ⇒ (S)S ⇒ (ε)S ⇒ ()S ⇒ ()(S)S ⇒ ()(ε)S ⇒ ()()S ⇒ ()()ε ⇒ ()()
```

Each step expands the leftmost remaining variable — exactly the kind of derivation GATE asks you to complete or verify.

Ambiguity

NOTES

A grammar is **ambiguous** if some string has **two different parse trees** (or leftmost derivations).

WORKED EXAMPLE

Worked numerical

Q: Grammar `E → E+E | E*E | id` . Show that "id+id*id" is ambiguous.
A: Two different parse trees exist:

```
Tree 1 (groups as id+(id*id)):        Tree 2 (groups as (id+id)*id):

      E                                       E
    / | \                                   / | \
   E  +  E                                 E  *  E
  id   / | \                               / | \  id
      E  *  E                           E  +  E
     id     id                         id     id
```

Both are valid parse trees for the same string using the same grammar → **ambiguous**. Fix: rewrite with precedence levels: `E → E+T | T, T → T*F | F, F → id` . This forces multiplication to bind tighter than addition, removing the ambiguity.

GATE FAVOURITE

GATE angle

Some languages are **inherently ambiguous** — every possible grammar for them is ambiguous, no rewrite fixes it (e.g. $\{a^i b^j c^k : i=j \text{ or } j=k\}$). GATE sometimes asks you to identify such a language rather than just spot ambiguity in one specific grammar.

Chomsky Normal Form (CNF)

NOTES

A CFG is in CNF if every rule looks like `A → BC` (two variables) or `A → a` (one terminal), with at most `S → ε` as an exception. Any CFG can be converted to an equivalent CNF grammar in three steps:

RULES & FACTS

1. Replace every terminal appearing alongside other symbols with a new variable (`a → Ca`).
2. Eliminate ε-productions and unit productions (`A → B`).
3. Break any RHS longer than 2 symbols into a chain of new binary variables.

WORKED EXAMPLE

Worked numerical — full CNF conversion

Q: Convert to CNF: `E → E+T | T, T → T*F | F, F → (E) | id` .
A:

```
Step 1 — terminal variables:
  C+ → +,  C* → *,  C( → (,  C) → )
  E → E C+ T | T
  T → T C* F | F
  F → C( E C) | id

Step 2 — eliminate unit productions (E→T, T→F):
  E → E C+ T | T C* F | C( E C) | id
  T → T C* F | C( E C) | id
  F → C( E C) | id

Step 3 — binarize every RHS longer than 2 symbols:
  "E C+ T" (3 symbols) → introduce X1 → C+ T,  giving E → E X1
  "T C* F" (3 symbols) → introduce X2 → C* F,  giving T → T X2
  "C( E C)" (3 symbols) → introduce Y1 → E C),  giving F → C( Y1

Final CNF grammar:
  E  → E X1 | T X2 | C( Y1 | id
  T  → T X2 | C( Y1 | id
  F  → C( Y1 | id
  X1 → C+ T
  X2 → C* F
  Y1 → E C)
  C+ → +     C* → *     C( → (     C) → )
```

Every rule now has exactly 2 variables on the RHS, or exactly 1 terminal — this is CNF. GATE often asks "how many new variables are introduced" (here: C+, C*, C(, C), X1, X2, Y1 = **7 new variables**) or "how many total productions does the CNF grammar have".

Derivations, parse trees & ambiguity counting

NOTES

A **leftmost** derivation always expands the leftmost variable; a **rightmost** derivation the rightmost. Every parse tree corresponds to exactly **one** leftmost and **one** rightmost derivation, so a grammar is ambiguous ⇔ some string has **two leftmost derivations** ⇔ two parse trees. For S → SS | a, the string a^n has Catalan(n−1) parse trees (n = 4 gives 5).

Simplifying a CFG

KEY POINT

Remove, **in this order**: (1) ε-productions → (2) unit productions (A → B) → (3) useless symbols (first non-generating, then unreachable). Another order can reintroduce unwanted productions. After this, convert to CNF or GNF.

Greibach Normal Form (GNF) & CYK

REFERENCE TABLE

Form	Production shape	Handy fact
CNF	A → BC or A → a	A string of length n ≥ 1 needs exactly **2n − 1** derivation steps
GNF	A → aα, a a terminal, α a (possibly empty) string of variables	Every step emits one terminal, so a string of length n takes exactly **n** steps; no left recursion

FORMULA / RULE

CYK algorithm decides w ∈ L(G) for a grammar in **CNF** in **$O(n^3 \cdot |G|)$** time. Table cell (i, l) = set of variables deriving the substring of length l starting at position i; combine two shorter cells using each rule A → BC. w is accepted iff the start symbol is in the top cell.

WORKED EXAMPLE

Worked numerical — CYK

Q: G: S → AB | BC, A → BA | a, B → CC | b, C → AB | a. Is w = baaba in L(G)?

A:

```
length 1:  b:{B}   a:{A,C}   a:{A,C}   b:{B}   a:{A,C}
length 2:  ba:{A,S}   aa:{B}   ab:{C,S}   ba:{A,S}
length 3:  baa:{ }   aab:{B}   aba:{B}
length 4:  baab:{ }   aaba:{A,C,S}
length 5:  baaba:{A,C,S}   → contains S
```

S is in the top cell, so baaba ∈ L(G).

4 Pushdown Automata

What is a PDA?

NOTES

A PDA is a 7-tuple `(Q, Σ, Γ, δ, q0, Z0, F)` : states Q, input alphabet Σ, **stack alphabet Γ**, transition function δ, start state q0, initial stack symbol Z0, and final states F. Each move can read an input symbol (or ε), pop the top stack symbol, and push a string of stack symbols — all in one step.

KEY POINT

Two ways to define "accept":

1. **By final state**: the PDA ends in a state in F (stack contents don't matter).
2. **By empty stack**: the PDA empties its entire stack (regardless of state).

These two definitions are provably **equivalent** in power — any PDA of one kind can be converted to the other.

Worked example: PDA for $a^n b^n$ (n ≥ 1)

KEY POINT

```
States: q0 (pushing), q1 (popping), qf (accept)
Stack alphabet: {a, Z0}

δ(q0, a, Z0) = (q0, aZ0)     -- push first 'a' on top of
bottom marker
δ(q0, a, a)  = (q0, aa)      -- push another 'a' for each
'a' read
δ(q0, b, a)  = (q1, ε)       -- first 'b' seen: pop one 'a',
switch to popping mode
δ(q1, b, a)  = (q1, ε)       -- pop one 'a' per 'b'
δ(q1, ε, Z0) = (qf, Z0)      -- all a's matched, bottom
marker exposed → accept
```

WORKED EXAMPLE

Worked trace

Q: Trace this PDA on input "aabb" and show the stack at each step.

A:

```
Step  Input read  State  Stack (top on left)
0     -           q0     Z0
1     a           q0     aZ0
2     a           q0     aaZ0
3     b           q1     aZ0      (popped one a)
4     b           q1     Z0       (popped the other a)
5     ε           qf     Z0       (Z0 exposed, moves to
accept state)
```

Input fully consumed, ended in accepting state qf → **"aabb" is accepted**. If a 5th symbol appeared before the stack was reduced to Z0, or if a 'b' arrived while the stack top was Z0 (no 'a' left to pop), the PDA would have no valid move and reject.

CFG ⇔ PDA equivalence

COMMON TRAP

Key theorem

A language is context-free **if and only if** some PDA accepts it. Every CFG can be mechanically converted into an equivalent PDA (simulate derivations using the stack to hold the "not yet expanded" part of the sentential form), and vice versa.

Deterministic vs Non-deterministic PDA

REFERENCE TABLE

	DPDA	NPDA
Power	Recognizes only **DCFLs** (deterministic CFLs) — a strict subset	Recognizes **all** CFLs
Example that needs NPDA	—	$\{ww^R : w \in (0+1)^*\}$ (even palindromes) — the PDA must "guess" the midpoint
Closure	DCFLs are closed under complement	CFLs are **not** closed under complement or intersection

WORKED EXAMPLE

Worked numerical

Q: Is the language $\{a^n b^n : n \geq 1\}$ deterministic context-free?

A: Yes — the PDA above never had to guess anything; at every step, the next move was uniquely determined by the current input symbol and stack top. It is a **DCFL**. Contrast with even palindromes $\{ww^R\}$, where the PDA cannot know where "w" ends and "w^R" begins without guessing — that language is context-free but **not** deterministic context-free.

5 Pumping Lemma & Language Properties

Pumping Lemma for Regular Languages

KEY POINT

If L is regular, there exists a **pumping length p** such that every string w ∈ L with |w| ≥ p can be split as `w = xyz` where:

1. |xy| ≤ p 2. |y| ≥ 1 (y is non-empty) 3. $xy^iz \in L$ for every i ≥ 0 (including i=0, "un-pumping").

NOTES

It's used as a **proof by contradiction**: assume L is regular (so p exists), pick a clever string, show that *every* way of splitting it fails condition 3 — contradiction, so L isn't regular.

WORKED EXAMPLE

Worked numerical

Q: Prove L = {0^n1^n : n ≥ 0} is not regular.

A:

```
Assume L is regular. Let p = pumping length.
Choose w = 0^p 1^p  (length 2p ≥ p, and w ∈ L). ✓ valid
choice

By the lemma, w = xyz with |xy| ≤ p and |y| ≥ 1.
Since |xy| ≤ p, xy lies entirely within the first p symbols
→ entirely within the 0's block.
So y = 0^k for some k ≥ 1 (y is made only of 0's).

Pump with i = 0:  xy⁰z = x z = 0^(p-k) 1^p
This has (p-k) zeros but p ones → p-k ≠ p since k ≥ 1 → NOT
of the form 0ⁿ1ⁿ.
xy⁰z ∉ L, contradicting condition 3.

∴ L is not regular.
```

Pumping Lemma for Context-Free Languages

KEY POINT

If L is context-free, there exists p such that every w ∈ L with |w| ≥ p can be split as `w = uvxyz` where:
1. |vxy| ≤ p 2. |vy| ≥ 1 (at least one of v,y is non-empty) 3. `uvⁱxyⁱz ∈ L` for every i ≥ 0.

WORKED EXAMPLE

Worked numerical

Q: Prove L = {$a^nb^nc^n$: n ≥ 0} is not context-free.

A:

```
Assume L is context-free. Let p = pumping length.
Choose w = a^p b^p c^p  (length 3p ≥ p, w ∈ L). ✓

By the lemma, w = uvxyz with |vxy| ≤ p and |vy| ≥ 1.
Since block boundaries are p apart and |vxy| ≤ p, the
substring "vxy"
can touch AT MOST 2 of the 3 blocks (a's, b's, c's) — it's
too short to
reach into all three.

Case: vxy touches only a's (or only b's, or only c's, or
straddles two
adjacent blocks) — in every case, pumping with i=2 (uv²xy²z)
increases
the count of symbols in AT MOST 2 of the 3 letter-blocks,
while the
third block's count stays the same.
→ the counts of a's, b's, c's can no longer all be equal.
→ uv²xy²z ∉ L, contradicting condition 3 for every possible
split.

∴ L is not context-free.
```

COMMON TRAP

Alternative technique: closure properties

You can also prove $a^nb^nc^n$ is not context-free **without** the pumping lemma, using closure properties: L1={$a^nb^nc^m$: n,m≥0} and L2={$a^mb^nc^n$: n,m≥0} are both context-free (simple PDAs exist for each). If $a^nb^nc^n$ were context-free, then since `L1 ∩ L2 = {aⁿbⁿcⁿ}`, and CFLs are **not closed under intersection** in general — this alone doesn't immediately give a contradiction, but combined with the pumping lemma result above, it illustrates exactly *why* CFLs aren't closed under intersection: the intersection of two CFLs can escape the class entirely.

CFL closure properties — the full picture

REFERENCE TABLE

Operation	Closed?
Union, concatenation, Kleene star, reversal	✔ Yes
Intersection with a **regular** language	✔ Yes
Intersection of two CFLs	✘ No
Complement	✘ No

Decision properties — quick facts

REFERENCE TABLE

Question	Decidable?
Is a given CFG's language empty?	✔ Decidable
Is a given CFG ambiguous?	✘ Undecidable
Do two CFGs generate the same language?	✘ Undecidable
Do two DFAs generate the same language?	✔ Decidable

COMMON TRAP

Pattern to notice

Almost everything about **finite automata / regular languages** is decidable (they're simple enough to fully analyze). The moment you ask an equivalence/ambiguity question about **grammars or Turing machines**, it usually becomes undecidable — Section 6 explains exactly why.

Master closure table — all language classes

REFERENCE TABLE

Operation	Regular	DCFL	CFL	Recursive	RE
Union	✔	✘	✔	✔	✔
Intersection	✔	✘	✘	✔	✔
Complement	✔	✔	✘	✔	✘
Concatenation	✔	✘	✔	✔	✔
Kleene star	✔	✘	✔	✔	✔
Reversal	✔	✘	✔	✔	✔
Difference $L_1 - L_2$	✔	✘	✘	✔	✘
Intersection with a regular language	✔	✔	✔	✔	✔
Homomorphism	✔	✘	✔	✘ (✔ if ε-free)	✔
Inverse homomorphism	✔	✔	✔	✔	✔

GATE FAVOURITE

GATE angle

Two reliable anchors: **DCFL** is closed under **complement** but not union/intersection; **RE** is closed under union and intersection but **not complement** (if both L and L^c are RE, L is recursive). Context-sensitive languages are closed under complement (Immerman–Szelepcsényi).

6 Turing Machines & Undecidability

What is a Turing Machine?

NOTES

A TM is a 7-tuple `(Q, Σ, Γ, δ, q0, B, F)` : states Q, input alphabet Σ, **tape alphabet Γ** (Σ ⊆ Γ, plus a blank symbol B), transition function `δ: Q×Γ → Q×Γ×{L,R}` , start state, blank symbol, final states. Unlike a PDA's stack, the TM has an **infinite tape** it can move left or right on and overwrite freely — this extra freedom makes it the most powerful model in the hierarchy.

KEY POINT

```
Tape:   ... [B][a][a][b][b][c][c][B] ...
                  ↑
             head here, current state q0
```

WORKED EXAMPLE

Worked example: deciding $a^n b^n c^n$

Q: Describe (informally) a TM algorithm that decides L = {$a^n b^n c^n$: n≥0}.
A:

```
Repeat:
  1. Scan right, find leftmost unmarked 'a', mark it (e.g.
replace with X). If no unmarked 'a' remains and no unmarked
b/c remains either → ACCEPT.
     If no unmarked 'a' but unmarked b or c still remain →
REJECT.
  2. Continue scanning right, find leftmost unmarked 'b',
mark it. If none exists → REJECT.
  3. Continue scanning right, find leftmost unmarked 'c',
mark it. If none exists → REJECT.
  4. Return head to the start, repeat from step 1.
```

This "cross off one of each, repeat, check all crossed off together" technique is the standard TM idea for matching counts across multiple blocks — exactly why a TM can decide this language even though no PDA (Part 4's pumping lemma proof) can.

TM variants & the Church-Turing Thesis

NOTES

Multi-tape TMs, nondeterministic TMs (NTMs), and TMs with a 2-D tape are all **equally powerful** as the standard single-tape TM — they can simulate each other (only with a difference in speed/efficiency, never in what's computable). The **Church-Turing Thesis** states: anything that is "intuitively/mechanically computable" is computable by some Turing Machine. It's a thesis (a belief backed by decades of evidence), not a provable theorem.

Recursive vs Recursively Enumerable (RE)

REFERENCE TABLE

Class	Definition
Recursive (decidable)	Some TM halts on **every** input, accepting strings in L and rejecting all others
Recursively enumerable (RE / recognizable)	Some TM halts and accepts every string in L, but may **loop forever** on strings not in L

COMMON TRAP

The theorem that ties them together

A language L is **recursive** if and only if **both L and its complement are RE**. So: recursive ⊂ RE (strictly) — every decidable language is recognizable, but some recognizable languages have a complement that isn't recognizable (they're "semi-decidable" — you can confirm membership but never confirm non-membership).

The Halting Problem

NOTES

HALT = { (M, w) : Turing machine M halts on input w }. This is the most famous undecidable problem — **no algorithm can decide it in general.**

NOTE

Proof sketch (diagonalization)

```
Suppose a TM H decides HALT: H(M,w) = accept if M halts on
w, reject otherwise.

Build a new TM D using H as a subroutine:
  D(M):  run H(M, M)
         if H says "accept" (M halts on itself) → D loops
forever
         if H says "reject" (M loops on itself)  → D halts
and accepts

Now ask: does D halt on input D itself?
  If D halts on D → H(D,D) must have said "reject" (that's
the only
    way D halts) → meaning D does NOT halt on D.
Contradiction.
  If D loops on D → H(D,D) must have said "accept" → meaning
D DOES
    halt on D. Contradiction.

Either way, contradiction → no such H can exist → HALT is
undecidable.
```

This is the same self-reference trick as Cantor's diagonalization / Russell's paradox — a recurring idea across CS theory.

Rice's Theorem — the fast way to prove undecidability

MUST REMEMBER

Any **non-trivial property of the language** recognized by a TM is undecidable. ("Non-trivial" = the property is true for some RE languages and false for others — not universally true or universally false.)

WORKED EXAMPLE

Worked numerical

Q: Classify each as decidable or undecidable: (a) Is L(M) empty? (b) Is L(M) = Σ*? (c) Does M have at least 50 states? (d) Does M halt on the empty string?
A:

```
(a) Is L(M) empty?          → UNDECIDABLE (non-trivial
property of L(M), by Rice's theorem)
(b) Is L(M) = Σ*?           → UNDECIDABLE (non-trivial
property of L(M), by Rice's theorem)
(c) Does M have ≥50 states?  → DECIDABLE (this is a property
of M's own description/structure,
                                NOT a property of the
language it recognizes — just count the
                                states listed. Rice's
theorem does not apply here.)
(d) Does M halt on ε?        → UNDECIDABLE (a specific
instance of the Halting Problem)
```

GATE FAVOURITE

GATE trap

Rice's theorem only applies to properties **of the language L(M)** — never to properties of the machine's own textual description (number of states, number of transitions, whether it uses a particular symbol). Those syntactic questions are usually trivially decidable — just read the machine's description.

Proving undecidability by reduction

NOTES

To prove a new problem P is undecidable: assume a decider for P exists, then use it to build a decider for a **known** undecidable problem (usually HALT) — a contradiction.

WORKED EXAMPLE

Worked numerical

Q: Prove `EMPTY = {M : L(M) = ∅}` is undecidable, by reduction from HALT.

A:

```
Suppose a decider R for EMPTY exists.
Given any instance (M, w) of HALT, construct M':
  M'(x): ignore x, simulate M on w.
         If M halts on w → M' accepts x. Else M' loops
forever.

L(M') = Σ* if M halts on w, or L(M') = ∅ if M never halts on
w.
So: M halts on w  ⇔  L(M') ≠ ∅  ⇔  R(M') says "not empty".

This means R could be used to decide HALT — but HALT is
undecidable.
Contradiction → no such R exists → EMPTY is undecidable.
(This is just Rice's theorem applied directly, worked out by
hand.)
```

Designing a TM — worked construction

KEY POINT

L = {$a^n b^n$: n ≥ 1}. Idea: mark the leftmost unmarked a with X, run right to the first unmarked b and mark it Y, return to the X, repeat; accept when only Y's remain.

```
δ(q0,a)=(q1,X,R)    δ(q0,Y)=(q3,Y,R)                    q0 =
find next a (or finish)
δ(q1,a)=(q1,a,R)    δ(q1,Y)=(q1,Y,R)    δ(q1,b)=(q2,Y,L)
q1 = scan right for a b
δ(q2,a)=(q2,a,L)    δ(q2,Y)=(q2,Y,L)    δ(q2,X)=(q0,X,R)
q2 = return to the X
δ(q3,Y)=(q3,Y,R)    δ(q3,□)=(qacc,□,R)
q3 = verify only Y's remain
(every missing transition = reject)
```

Running time is Θ(n^2): the machine accepts $a^n b^n$ in exactly **$2n^2 + 2n + 1$** moves (ab → 5, aabb → 13, aaabbb → 25).

Universal TM, enumerators & PCP

KEY POINT

Universal TM (UTM): takes ⟨M, w⟩ as input and simulates M on w; its language A_{TM} is **RE but not recursive**, and its complement is not RE.
Enumerator: a TM that prints strings; a language is RE ⇔ some enumerator prints exactly its strings.
Post Correspondence Problem (PCP): given dominoes (x_i / y_i), is there a sequence $i_1 \ldots i_k$ with $x_{i1} \ldots x_{ik} = y_{i1} \ldots y_{ik}$? **Undecidable** (for alphabets of size ≥ 2). It is the standard tool for showing that CFG ambiguity and "$L(G_1) \cap L(G_2) = \emptyset$" are undecidable.

```
Dominoes: 1 = a/ab, 2 = b/ca, 3 = ca/a, 4 = abc/c.
Sequence 1,2,3,1,4:
top    = a · b · ca · a · abc = abcaaabc
bottom = ab · ca · a · ab · c = abcaaabc   ✓ match
```

KEY POINT

Linear Bounded Automata / CSLs: every CSL is **decidable** (recursive), so CSL ⊊ Recursive ⊊ RE. For an LBA, *"does M accept w?"* is decidable, but *"is L(M) empty?"* is undecidable.

The Chomsky Hierarchy — the full picture

REFERENCE TABLE

Type	Grammar	Recognized by	Language class
Type 3	Regular grammar	DFA / NFA	Regular languages
Type 2	Context-free grammar	PDA	Context-free languages
Type 1	Context-sensitive grammar	Linear Bounded Automaton (LBA)	Context-sensitive languages
Type 0	Unrestricted grammar	Turing Machine	Recursively enumerable

NOTES

Each class is a **strict subset** of the next: `Regular ⊊ Context-Free ⊊ Context-Sensitive ⊊ Recursively-Enumerable`. This single hierarchy is the backbone connecting every part of this site — worth memorizing cold.

7 Problem → Standard Method Playbook

METHOD

Moore vs Mealy — output questions

Recognition: the question gives a finite-state machine and asks about output timing, number of states, or conversion.

- **Moore:** output depends only on the current state; output changes after entering a state. For n states, at most n distinct state outputs.
- **Mealy:** output depends on current state and input; output is attached to transitions. A Mealy machine can often implement the same behaviour with fewer states.
- For Mealy → Moore, split a state when different incoming transitions require different output labels; this can increase the state count.

METHOD

CFG ambiguity — standard test

Recognition: the question asks whether a grammar is ambiguous or asks for two derivations of a string.

1. Find a candidate string that admits two distinct parse trees.
2. Prefer short strings generated through two competing productions, such as expression grammars without precedence rules.
3. Show two distinct **leftmost derivations** or two parse trees. Two different rightmost derivations alone are not sufficient unless they correspond to different parse trees.

GATE trap

Ambiguity is a property of a grammar, not merely of a language. A language can be inherently ambiguous, but a particular ambiguous grammar may have an equivalent unambiguous grammar.

METHOD

CYK parsing — recognition method

CYK requires the grammar in **Chomsky Normal Form** (plus the usual handling of ε). For a string of length n, build a triangular table. Cell T[i,j] stores variables that derive substring w[i…j].

1. Length 1: put variables generating the terminal.
2. For each longer span, try every split k.
3. If B ∈ T[i,k] and C ∈ T[k+1,j], add A whenever A → BC.
4. Accept iff the start variable is in the full-span cell.

Standard time complexity: **O(n³\|G\|)** for a fixed grammar, commonly stated as O(n³).

METHOD

Greibach Normal Form — conversion anchor

GNF productions have the form `A → aα`, where `a` is a terminal and α is a (possibly empty) sequence of variables. Consequently, a derivation of a non-empty string of length n uses exactly **n production steps** in GNF.

Do not confuse this with CNF: CNF uses `A → BC` or `A → a` and is convenient for CYK; GNF starts every production with a terminal and is useful for PDA/derivation reasoning.

METHOD

TM construction — marking / matching template

For languages such as `{aⁿbⁿ}`, `{aⁿbⁿcⁿ}`, or equal-count constraints:

1. Mark one unmatched symbol from the left block.
2. Scan right to find and mark its matching symbol in the next block.
3. Return to the left and repeat.
4. When the source block is exhausted, make a final pass to verify that no unmatched symbols remain.
5. Reject immediately if a required matching symbol is missing or appears in the wrong order.

WORKED EXAMPLE

Worked problem — CYK

Grammar: S → AB | BC, A → BA | a, B → CC | b, C → AB | a. Determine whether `ba` is generated.

```
Length 1:
T[1,1] for 'b' = {B}
T[2,2] for 'a' = {A,C}

Length 2, split after position 1:
(B, A) gives no S because neither S→BA nor S→BC with
second=A.
(B, C) gives S→BC, so S ∈ T[1,2].
Therefore 'ba' ∈ L(G).
```

WORKED EXAMPLE

Worked problem — Moore/Mealy state reasoning

Question: A Mealy machine has 4 states and is converted to a Moore machine by splitting states according to distinct output labels on incoming transitions. If the four states have respectively 1, 2, 1, and 3 distinct incoming output labels, what is the resulting Moore state count before unreachable-state removal?

```
Each Mealy state becomes one Moore state per distinct
incoming output label:
1 + 2 + 1 + 3 = 7 states.
Answer: 7 before removing unreachable states.
```

8 Last-minute one-liners

LAST-MINUTE · FORMULAS & RULES

- **CNF:** |w| = n takes 2n−1 steps. GNF: n steps. CYK needs CNF and runs in O(n³).

LAST-MINUTE · MUST REMEMBER

- **Moore:** output is associated with the STATE; Mealy: output is associated with the TRANSITION (state + input). Mealy→Moore can add states. Never fix an output count from the machine type alone — it depends on the question's output convention (is the start state's output printed?) and the input length.
- PCP is undecidable; it proves CFG ambiguity and CFG-intersection emptiness undecidable. LBA acceptance is decidable, LBA emptiness is not.
- DFA and NFA recognise exactly the regular languages; nondeterminism does not increase expressive power here.
- Every DFA is an NFA, but an NFA can be exponentially more compact than an equivalent DFA.
- Regular languages are closed under union, intersection, complement, concatenation, Kleene star, and difference.
- CFGs generate context-free languages; parse trees and derivations describe how strings are generated.
- PDA recognises exactly the context-free languages under the standard acceptance model.
- Regular languages are a proper subset of context-free languages; e.g. $\{a^n b^n \mid n \geq 0\}$ is CFL but not regular.
- For CFL pumping lemma, the pumped portions are constrained to lie within a bounded window around a decomposition of the string.
- A Turing machine is more powerful than DFA/PDA and can model general algorithmic computation.
- GNF productions begin with a terminal; a length-n nonempty string has an n-step leftmost derivation in GNF.
- **TM matching template:** mark one symbol, scan for its mate, return, repeat, then verify no leftovers.

LAST-MINUTE · COMMON TRAPS

- Regular closure covers FINITE operations only — infinite unions and arbitrary subsets of regular languages need not be regular.
- Closure under an operation does not mean a particular construction is identical for every representation; use the appropriate automaton/product/complement construction.
- Pumping lemma is mainly a tool to prove non-membership/non-regularity; satisfying the pumping lemma does not by itself prove regularity.
- **Reduction direction matters:** to prove B hard/undecidable using known-hard A, reduce A to B.
- Ambiguous grammar ≠ inherently ambiguous language; prove grammar ambiguity by two distinct parse trees/leftmost derivations for one string.

LAST-MINUTE · SHORTCUTS

- **CFG simplification order:** ε-productions → unit productions → useless symbols.

LAST-MINUTE · GATE FAVOURITES

- **DCFL:** closed under complement and inverse homomorphism; NOT union, intersection, concatenation, star, reversal.
- Ambiguous grammar means some string has more than one parse tree/leftmost derivation/rightmost derivation.
- Decidable means some TM halts on every input and answers correctly; recognizable/RE only requires halting and accepting members.
- **CYK:** convert/assume CNF, fill substrings bottom-up, and accept iff start variable appears in the full-span cell.